

CS2610: Computer Organization and Architecture Lab

Lab 9: Paging-II

16th March 2026

In this lab, we will look at an Sv-39 virtualization mechanism and enable paging support to it. Consider the following mapping of pages as given in the below diagrams, as well as the virtual address view of the same.

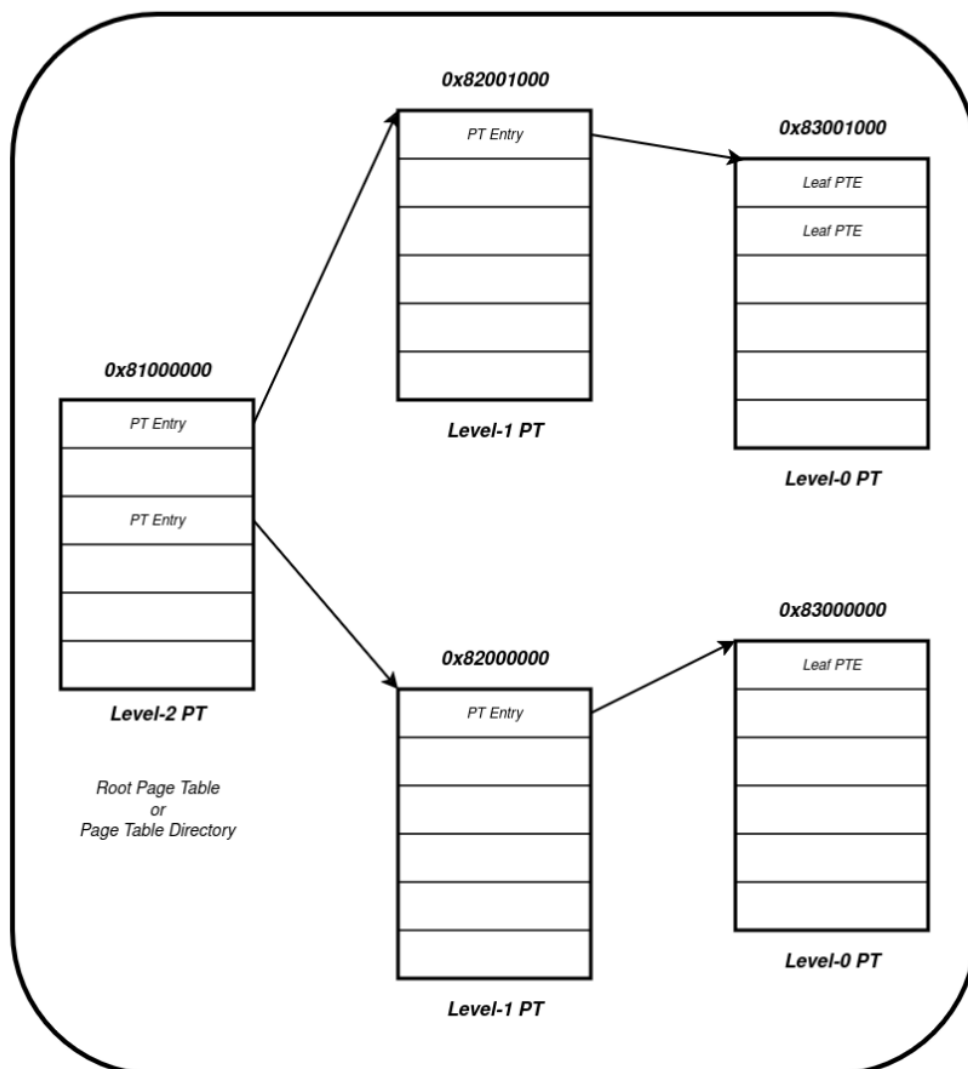


Figure 1: Page Table Structure for Sv39

Here, the entry point of main is 0x80000000 (as given in the linker file).

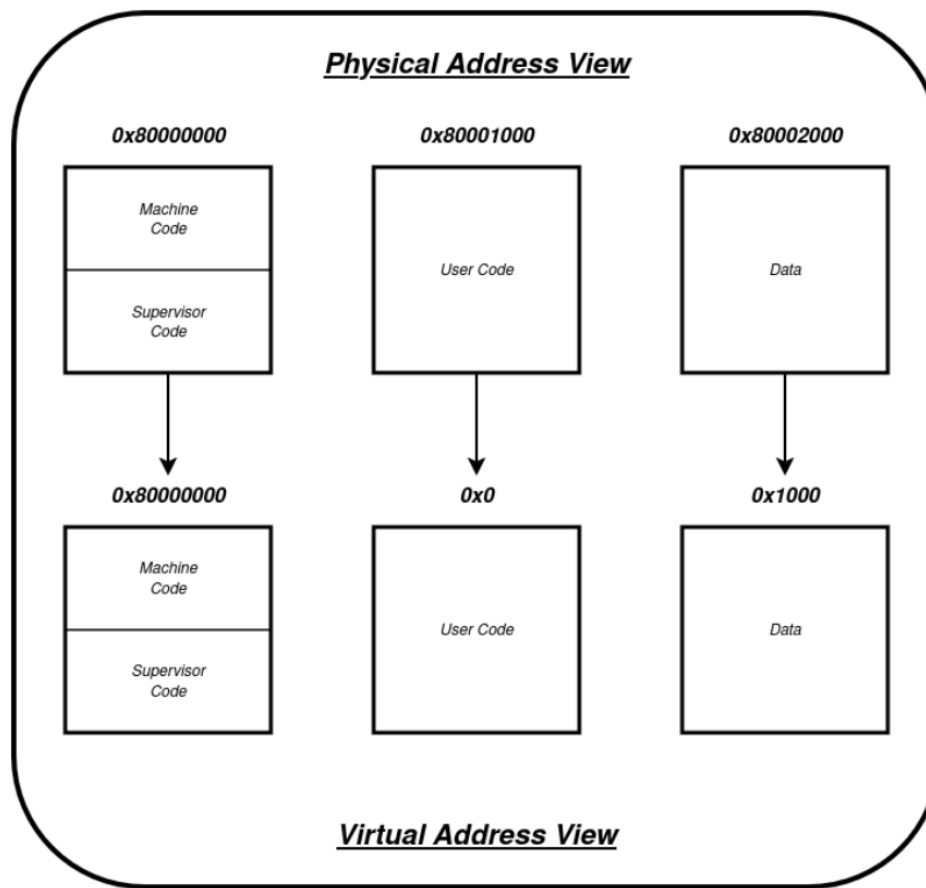


Figure 1: Virtual Address View

- The user code and user data lie in the pages beginning from the physical address 0x80001000 and 0x80002000 respectively.
- The page table is initialized with the mapping of virtual address 0x00000000 to the user level code section and 0x00001000 to the data section.

The page table entries are set as per the diagrams given. Now, we move on to handling both instruction and data page faults in this setup. The handling should be done as follows:

- **Instruction page fault:** Allot an available physical page and swap in the code. Assume every instruction page contains the same code as user code. Hence you will have to copy the contents of the page beginning from 0x80001000 into the new physical page. (refer Figure 2)
- **Data page fault:** Map the fault-generating virtual page to the physical page of the user data page starting from address 0x80002000. (refer Figure 3)
- Handle both the Level 0 page table entry indices and Level 1 page table indices dynamically in both the cases. DO NOT hardcode the values. Your code should be able to handle any page-faults generated by the user code in the virtual space 0x00000000 to 0x00400000.

1 Submission Instruction

1. Your assembly code file

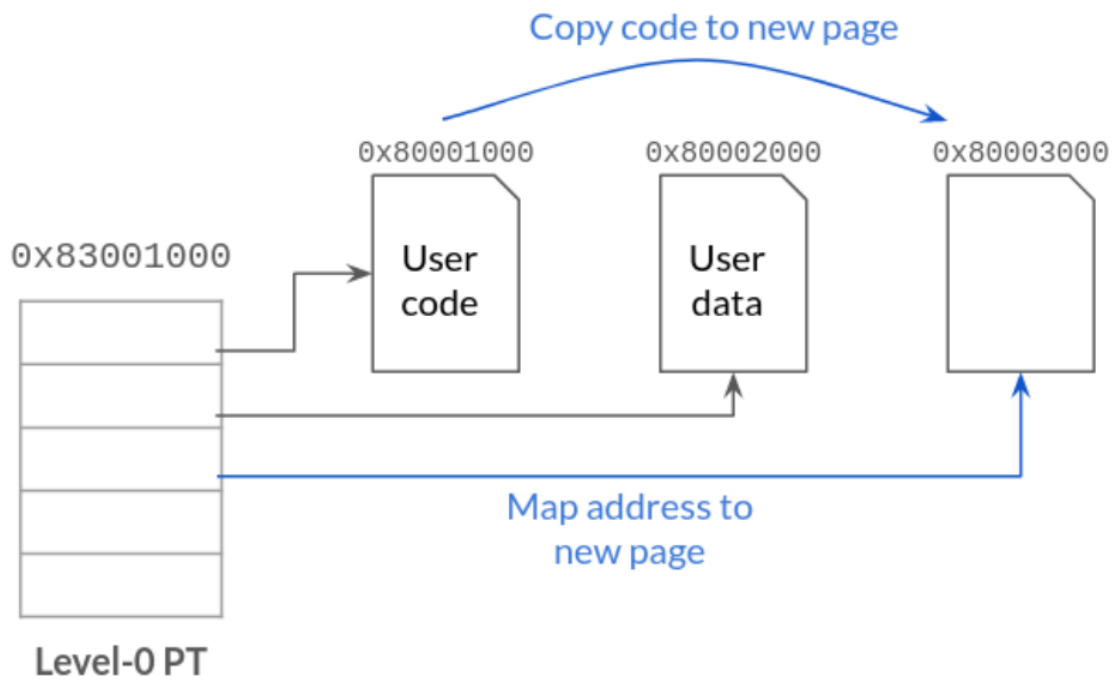


Figure 2: On an instruction page fault

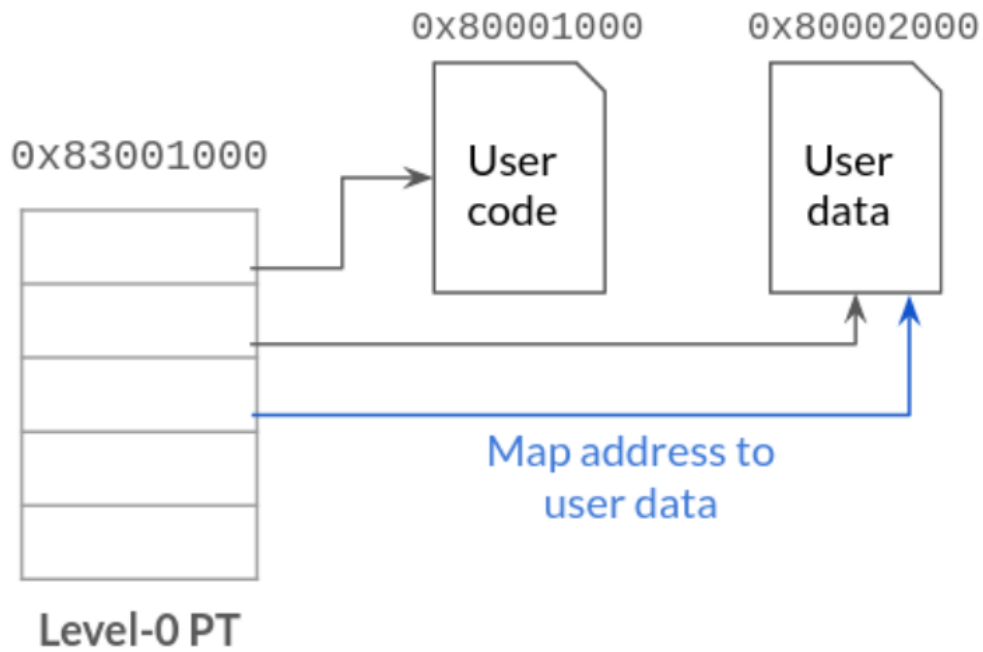


Figure 3: On a store page fault

2. The corresponding dump file
3. Screenshots of spike simulation showing, for each instruction page fault for instruction and data page faults, with one example within virtual address 0x200000 and one example for a virtual address greater than 0x200000. (a) address of var count stored in register t1 (b) value of var count stored in register t2 (c) jump address stored in register t3